

Introduction:

parser.c is a basic parser and interpreter for a mini programming language. This language supports integer declarations, arithmetic operations, and print statements.

Users can declare integer variables with single-character identifiers like x or multi-character identifiers such as total. During assignment, the program accepts both single-digit and multi-digit integer values, such as 5 and 20.

The program reads a source file given in the command line. It tokenizes the input, parses it using a context-free grammar (CFG), checks for syntax and semantic errors, and executes the valid logic to produce the correct output.

1) CFG

$\langle \text{program} \rangle \rightarrow \langle \text{statement_list} \rangle$
 $\langle \text{statement_list} \rangle \rightarrow \langle \text{statement} \rangle \langle \text{statement_list} \rangle \mid \epsilon$
 $\langle \text{statement} \rangle \rightarrow \langle \text{assign} \rangle \mid \langle \text{print_stmt} \rangle$
 $\langle \text{assign} \rangle \rightarrow \text{int id} = \langle \text{expr} \rangle ;$
 $\langle \text{print_stmt} \rangle \rightarrow \text{print} (\langle \text{id} \rangle) ;$
 $\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \langle \text{factor} \rangle$
 $\langle \text{factor} \rangle \rightarrow + \langle \text{term} \rangle \langle \text{factor} \rangle \mid \epsilon$
 $\langle \text{term} \rangle \rightarrow \langle \text{id} \rangle \mid \langle \text{number} \rangle$
 $\langle \text{id} \rangle \rightarrow \text{letter} (\text{letter} \mid \text{digit})^*$
 $\langle \text{number} \rangle \rightarrow \text{digit}^+$

2) Tokenisation

Input	Token
int	TOKEN_INT
print	TOKEN_PRINT
x y z total	TOKEN_ID
5 20	TOKEN_NUMBER
=	TOKEN_ASSIGN
+	TOKEN_PLUS
;	TOKEN_SEMI
(	TOKEN_LPAREN
)	TOKEN_RPAREN

The tokeniser reads the input one character at a time. It classifies alphabetic sequences as keywords or identifiers, numeric sequences as numbers, and single-character symbols according to their token types. Whitespace characters are ignored during tokenization.

3) Parser Design

Each grammar rule is implemented as a separate function, following the recursive-descent parsing technique.

CFG Rule	Parser Function
<program>	program()
<statement_list>	statement_list()
<statement>	statement()
<assign>	assign()
<expr>	expr()
<factor>	factor()
<term>	term()
<print_stmt>	print_stmt()
<id>	TOKEN_ID (handled by tokenizer)
<number>	TOKEN_NUMBER (handled by tokeniser)

4) Syntax error handling

Syntax errors occur when the current token does not match the expected token according to the grammar rule. The match() function validates tokens and stops execution right away when it finds an error.

Ex: Missing semicolon, missing parentheses, Invalid statemen start, Incorrect expression format

Semantic error handling

A symbol table is maintained to store declared variables and their values.

Semantic errors such as redeclaration of variables and usage of undeclared variables are detected during parsing and expression evaluation.

Ex: int x =5;
int x= 10;

5) Arithmetic expression evaluation & Execution

Arithmetic expressions are evaluated during parsing using inherited values.

The print statement retrieves the value of the variable from the symbol table and displays it as output.

The parser explicitly checks for invalid starting tokens and handles end-of-file as a valid termination condition to prevent false syntax errors after successful parsing.

6) Example screenshots:

6.1) With correct input file

The screenshot shows a C program named `parser.c` and an input file `inputfile.txt`. The C program includes `<stdio.h>`, `<stdlib.h>`, `<ctype.h>`, and `<string.h>`. It defines an enum `TokenType` with values `TOKEN_INT`, `TOKEN_PRINT`, `TOKEN_ID`, `TOKEN_NUMBER`, `TOKEN_ASSIGN`, `TOKEN_PLUS`, `TOKEN_SEMI`, `TOKEN_LPAREN`, `TOKEN_RPAREN`, `TOKEN_EOF`, and `TOKEN_INVALID`. The input file contains the code: `int y = 5;`, `int x = 20;`, `int z = x + y;`, and `print(z);`. The terminal output shows the command `gcc parser.c -o parser` and `./parser inputfile.txt` being executed successfully.

6.2) Syntax errors

6.2.1 Missing Semicolon

The screenshot shows the same C program `parser.c` and input file `inputfile.txt`. The input file now contains `int x = 10` on line 1 and `print(x);` on line 2, but it is missing a semicolon at the end of line 1. The terminal output shows the command `gcc parser.c -o parser` and `./parser inputfile.txt` being executed, resulting in a `Syntax Error`.

6.2.2 Invalid Statement

The screenshot shows the same C program `parser.c` and input file `inputfile.txt`. The input file now contains `x = 5;` on line 1 and an empty line 2. The terminal output shows the command `gcc parser.c -o parser` and `./parser inputfile.txt` being executed, resulting in a `Syntax Error: Invalid start of program`.

6.2.3 Missing Parentheses

The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6
7 //token types
8 typedef enum {
9     TOKEN_INT,
10    TOKEN_PRINT,
11    TOKEN_ID,
12    TOKEN_NUMBER,
13    TOKEN_ASSIGN,
14    TOKEN_PLUS,
15    TOKEN_SEMI,
16    TOKEN_LPAREN,
17    TOKEN_RPAREN,
18    TOKEN_EOF,
19    TOKEN_INVALID
20 } TokenType;
21
```

The `inputfile.txt` file contains the following code:

```
1 int x = 5;
2 print(x;
3
```

The terminal shows the following commands and output:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> gcc parser.c -o parser
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> ./parser inputfile.txt
Syntax Error
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment>
```

6.2.4 Invalid Expression

The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6
7 //token types
8 typedef enum {
9     TOKEN_INT,
10    TOKEN_PRINT,
11    TOKEN_ID,
12    TOKEN_NUMBER,
13    TOKEN_ASSIGN,
14    TOKEN_PLUS,
15    TOKEN_SEMI,
16    TOKEN_LPAREN,
17    TOKEN_RPAREN,
18    TOKEN_EOF,
19    TOKEN_INVALID
20 } TokenType;
21
```

The `inputfile.txt` file contains the following code:

```
1 int x = + 5;
2
```

The terminal shows the following commands and output:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> gcc parser.c -o parser
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> ./parser inputfile.txt
Syntax Error in expression
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment>
```

6.2.5 Missing assignment operator

The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6
7 //token types
8 typedef enum {
9     TOKEN_INT,
10    TOKEN_PRINT,
11    TOKEN_ID,
12    TOKEN_NUMBER,
13    TOKEN_ASSIGN,
14    TOKEN_PLUS,
15    TOKEN_SEMI,
16    TOKEN_LPAREN,
17    TOKEN_RPAREN,
18    TOKEN_EOF,
19    TOKEN_INVALID
20 } TokenType;
21
```

The `inputfile.txt` file contains the following code:

```
1 int x 5;
2
```

The terminal shows the following commands and output:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> gcc parser.c -o parser
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment> ./parser inputfile.txt
Syntax Error
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Models and Programming Language Concepts\Assignment>
```

6.2.6 Invalid Token

The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains a token type enum and a main function that reads tokens from `inputfile.txt`. The `inputfile.txt` file contains the following code:

```
1 int x = 5;  
2 int y = 2 @ 3;  
3
```

The terminal output shows the following commands and results:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> gcc parser.c -o parser  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> ./parser inputfile.txt  
Syntax Error  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment>
```

6.3) Semantic Errors

6.3.1 Variable not declared

The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains a token type enum and a main function that reads tokens from `inputfile.txt`. The `inputfile.txt` file contains the following code:

```
1 int x = y + 5;  
2
```

The terminal output shows the following commands and results:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> gcc parser.c -o parser  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> ./parser inputfile.txt  
Semantic Error: Variable not declared  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment>
```

6.3.2 Redeclaration

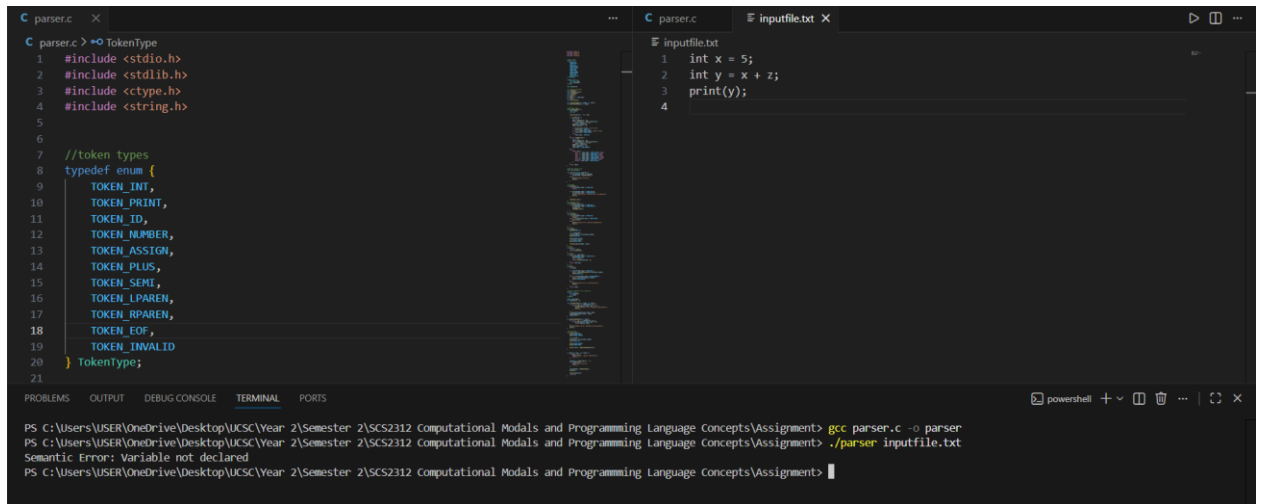
The screenshot shows a C++ IDE with two files: `parser.c` and `inputfile.txt`. The `parser.c` file contains a token type enum and a main function that reads tokens from `inputfile.txt`. The `inputfile.txt` file contains the following code:

```
1 int x = 5;  
2 int x = 10;  
3
```

The terminal output shows the following commands and results:

```
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> gcc parser.c -o parser  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> ./parser inputfile.txt  
Semantic Error: Variable redeclared  
PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment>
```

6.4) Mixed errors



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <ctype.h>
4 #include <string.h>
5
6
7 //token types
8 typedef enum {
9     TOKEN_INT,
10    TOKEN_PRINT,
11    TOKEN_ID,
12    TOKEN_NUMBER,
13    TOKEN_ASSIGN,
14    TOKEN_PLUS,
15    TOKEN_SEMI,
16    TOKEN_LPAREN,
17    TOKEN_RPAREN,
18    TOKEN_EOF,
19    TOKEN_INVALID
20 } TokenType;
21
```

```
1 int x = 5;
2 int y = x + z;
3 print(y);
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> gcc parser.c -o parser

PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> ./parser inputfile.txt

Semantic Error: Variable not declared

PS C:\Users\USER\OneDrive\Desktop\UCSC\Year 2\Semester 2\SCS2312 Computational Modals and Programming Language Concepts\Assignment> |